

A01:2025 Broken Access Control



Background.

Maintaining its position at #1 in the Top Ten, 100% of the applications tested were found to have some form of broken access control. Notable CWEs included are *CWE-200: Exposure of Sensitive Information to an Unauthorized Actor*, *CWE-201: Exposure of Sensitive Information Through Sent Data*, *CWE-918 Server-Side Request Forgery (SSRF)*, and *CWE-352: Cross-Site Request Forgery (CSRF)*. This category has the highest number of occurrences in the contributed data, and second highest number of related CVEs.

Score table.

CWEs Mapped	Max Incidence Rate	Avg Incidence Rate	Max Coverage	Avg Coverage	Avg Weighted Exploit	Avg Weighted Impact
40	20.15%	3.74%	100.00%	42.93%	7.04	3.84

Description.

Access control enforces policy such that users cannot act outside of their intended permissions. Failures typically lead to unauthorized information disclosure, modification or destruction of all data, or performing a business function outside the user's limits. Common access control vulnerabilities include:

- Violation of the principle of least privilege, commonly known as deny by default, where access should only be granted for particular capabilities, roles, or users, but is available to anyone.
- Bypassing access control checks by modifying the URL (parameter tampering or force browsing), internal application state, or the HTML page, or by using an attack tool that modifies API requests.
- Permitting viewing or editing someone else's account by providing its unique identifier (insecure direct object references)
- An accessible API with missing access controls for POST, PUT, and DELETE.

- Elevation of privilege. Acting as a user without being logged in or or gaining privileges beyond those expected of the logged in user (e.g. admin access).
- Metadata manipulation, such as replaying or tampering with a JSON Web Token (JWT) access control token, a cookie or hidden field manipulated to elevate privileges, or abusing JWT invalidation.
- CORS misconfiguration allows API access from unauthorized or untrusted origins.
- Force browsing (guessing URLs) to authenticated pages as an unauthenticated user or to privileged pages as a standard user.

How to prevent.

Access control is only effective when implemented in trusted server-side code or serverless APIs, where the attacker cannot modify the access control check or metadata.

- Except for public resources, deny by default.
- Implement access control mechanisms once and reuse them throughout the application, including minimizing Cross-Origin Resource Sharing (CORS) usage.
- Model access controls should enforce record ownership rather than allowing users to create, read, update, or delete any record.
- Unique application business limit requirements should be enforced by domain models.
- Disable web server directory listing and ensure file metadata (e.g., .git) and backup files are not present within web roots.
- Log access control failures, alert admins when appropriate (e.g., repeated failures).
- Implement rate limits on API and controller access to minimize the harm from automated attack tooling.
- Stateful session identifiers should be invalidated on the server after logout. Stateless JWT tokens should be short-lived to minimize the window of opportunity for an attacker. For longer-lived JWTs, consider using refresh tokens and following OAuth standards to revoke access.
- Use well-established toolkits or patterns that provide simple, declarative access controls.

Developers and QA staff should include functional access control in their unit and integration tests.

Example attack scenarios.

Scenario #1: The application uses unverified data in an SQL call that is accessing account information:

```
pstmt.setString(1, request.getParameter("acct"));  
ResultSet results = pstmt.executeQuery( );
```

An attacker can simply modify the browser's 'acct' parameter to send any desired account number. If not correctly verified, the attacker can access any user's account.

```
https://example.com/app/accountInfo?acct=notmyacct
```

Scenario #2: An attacker simply forces browsers to target URLs. Admin rights are required for access to the admin page.

```
https://example.com/app/getappInfo  
https://example.com/app/admin_getappInfo
```

If an unauthenticated user can access either page, it's a flaw. If a non-admin can access the admin page, this is a flaw.

Scenario #3: An application puts all of their access control in their front-end. While the attacker cannot get to `https://example.com/app/admin_getappInfo` due to JavaScript code running in the browser, they can simply execute:

```
$ curl https://example.com/app/admin_getappInfo
```

from the command line.

References.

- [OWASP Proactive Controls: C1: Implement Access Control](#)
- [OWASP Application Security Verification Standard: V8 Authorization](#)
- [OWASP Testing Guide: Authorization Testing](#)
- [OWASP Cheat Sheet: Authorization](#)
- [PortSwigger: Exploiting CORS misconfiguration](#)
- [OAuth: Revoking Access](#)

List of Mapped CWEs

- [CWE-22 Improper Limitation of a Pathname to a Restricted Directory \('Path Traversal'\)](#)
- [CWE-23 Relative Path Traversal](#)
- [CWE-36 Absolute Path Traversal](#)
- [CWE-59 Improper Link Resolution Before File Access \('Link Following'\)](#)

- CWE-61 UNIX Symbolic Link (Symlink) Following
- CWE-65 Windows Hard Link
- CWE-200 Exposure of Sensitive Information to an Unauthorized Actor
- CWE-201 Exposure of Sensitive Information Through Sent Data
- CWE-219 Storage of File with Sensitive Data Under Web Root
- CWE-276 Incorrect Default Permissions
- CWE-281 Improper Preservation of Permissions
- CWE-282 Improper Ownership Management
- CWE-283 Unverified Ownership
- CWE-284 Improper Access Control
- CWE-285 Improper Authorization
- CWE-352 Cross-Site Request Forgery (CSRF)
- CWE-359 Exposure of Private Personal Information to an Unauthorized Actor
- CWE-377 Insecure Temporary File
- CWE-379 Creation of Temporary File in Directory with Insecure Permissions
- CWE-402 Transmission of Private Resources into a New Sphere ('Resource Leak')
- CWE-424 Improper Protection of Alternate Path
- CWE-425 Direct Request ('Forced Browsing')
- CWE-441 Unintended Proxy or Intermediary ('Confused Deputy')
- CWE-497 Exposure of Sensitive System Information to an Unauthorized Control Sphere
- CWE-538 Insertion of Sensitive Information into Externally-Accessible File or Directory
- CWE-540 Inclusion of Sensitive Information in Source Code
- CWE-548 Exposure of Information Through Directory Listing
- CWE-552 Files or Directories Accessible to External Parties
- CWE-566 Authorization Bypass Through User-Controlled SQL Primary Key
- CWE-601 URL Redirection to Untrusted Site ('Open Redirect')
- CWE-615 Inclusion of Sensitive Information in Source Code Comments
- CWE-639 Authorization Bypass Through User-Controlled Key
- CWE-668 Exposure of Resource to Wrong Sphere
- CWE-732 Incorrect Permission Assignment for Critical Resource
- CWE-749 Exposed Dangerous Method or Function
- CWE-862 Missing Authorization

- [CWE-863 Incorrect Authorization](#)
- [CWE-918 Server-Side Request Forgery \(SSRF\)](#)
- [CWE-922 Insecure Storage of Sensitive Information](#)
- [CWE-1275 Sensitive Cookie with Improper SameSite Attribute](#)